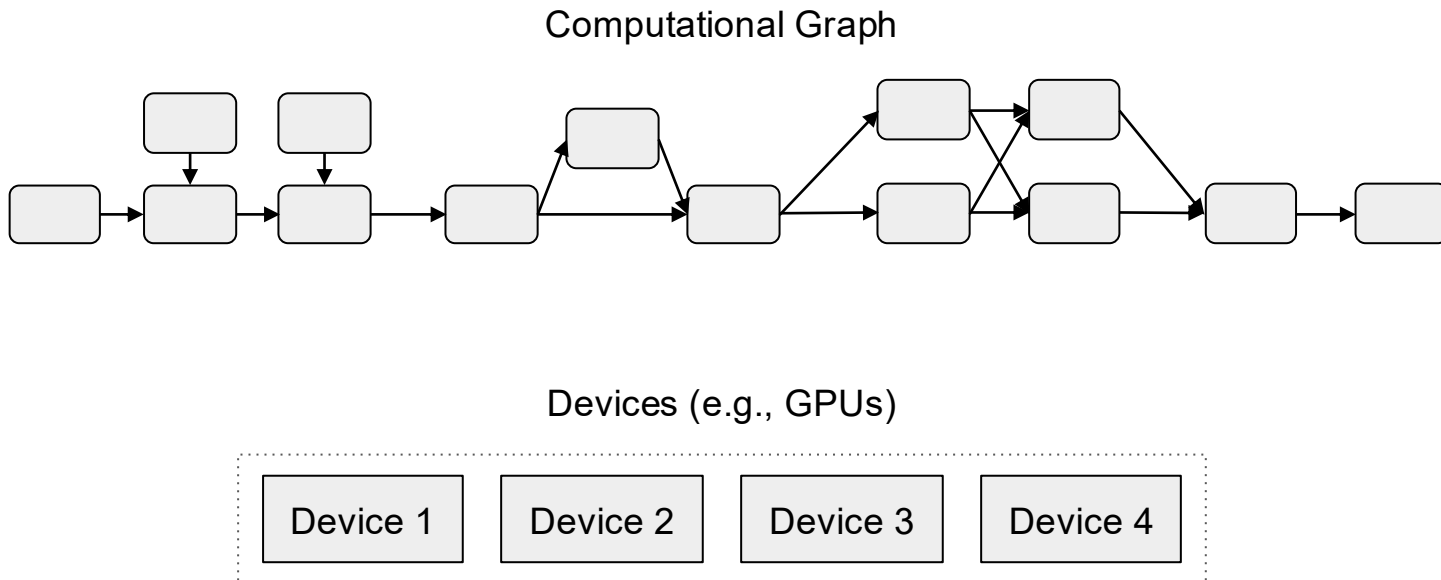


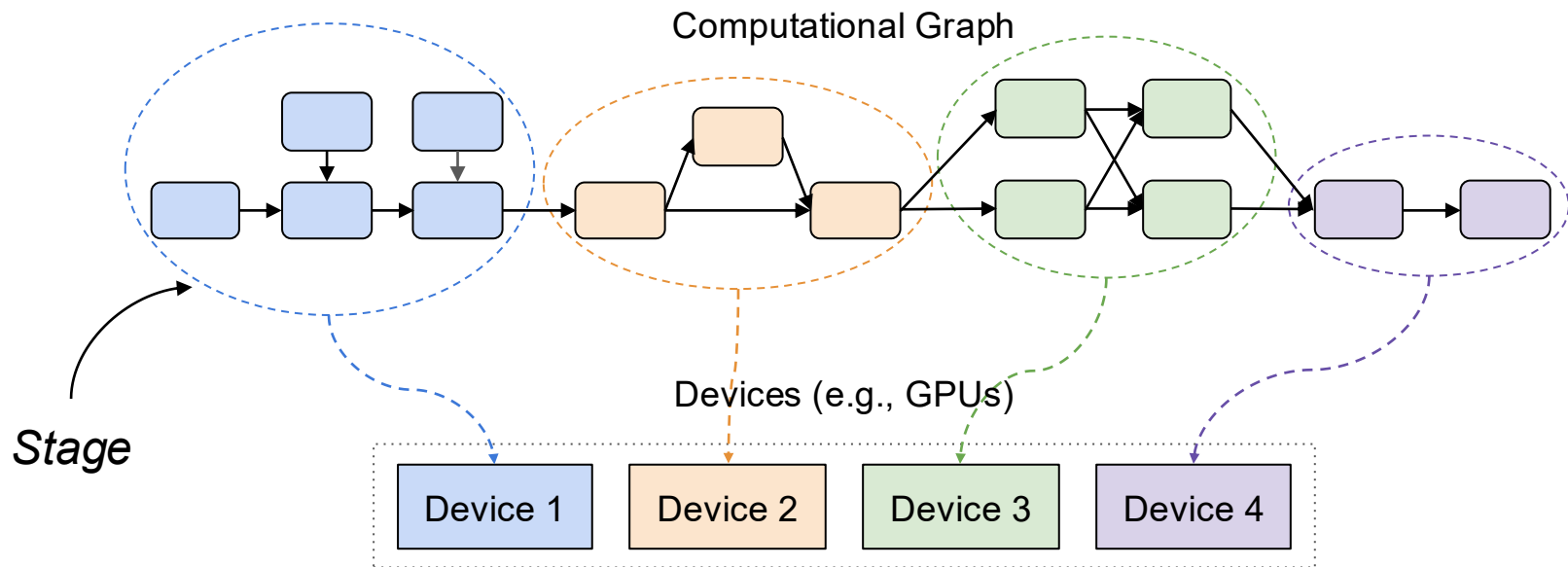
Where we are

- Why Parallelization: Technology Trend
- ML Parallelism Overview
- Collective Communication Overview
- **Inter and intra-op parallelism**
- **Summary of the course**

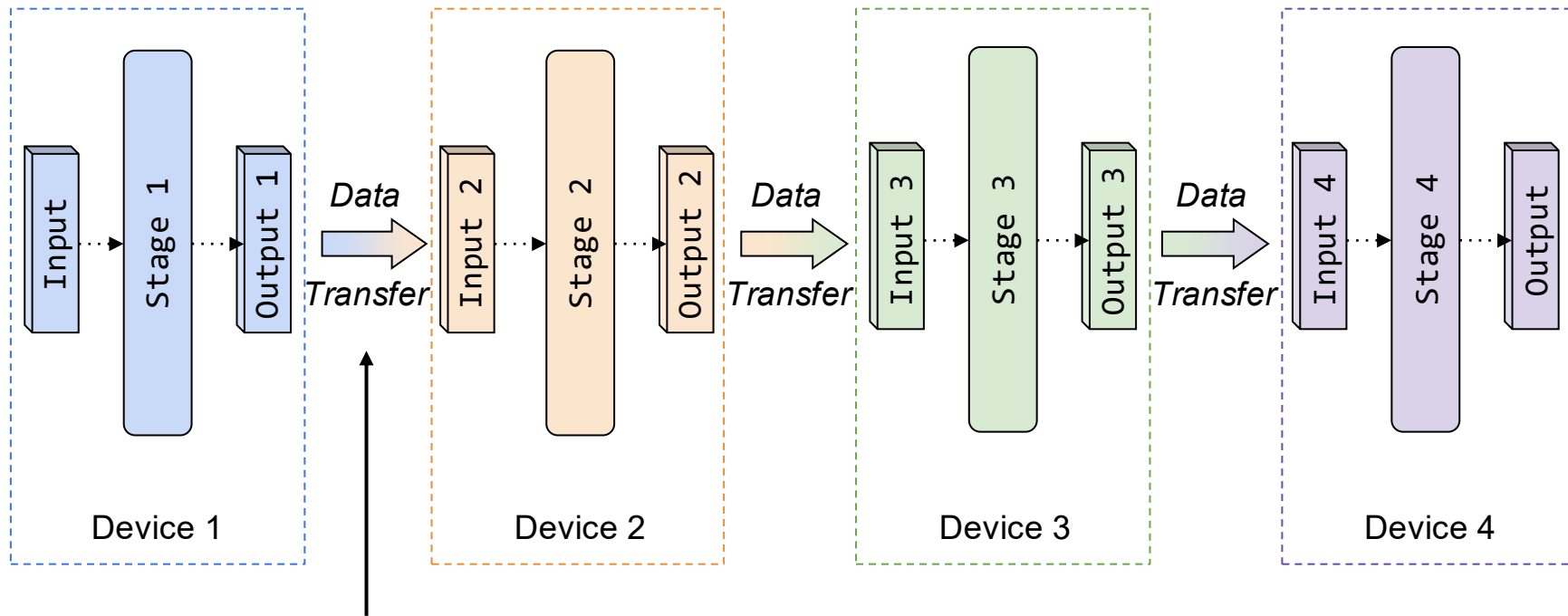
Computational Graph (Neural Networks) → Stages



Computational Graph (Neural Networks) → Stages

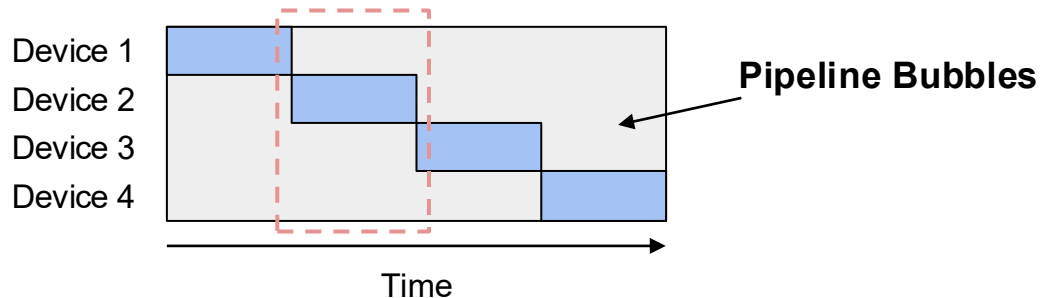


Execution & Data Movement



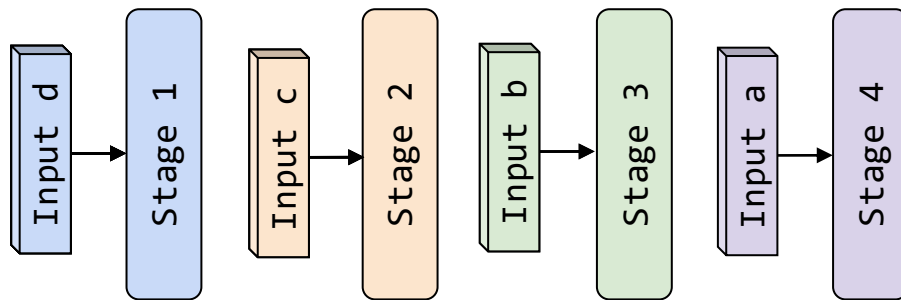
Note: The time spent on data transfer is typically **small**, since we only communicate stage outputs at stage boundaries between two stages.

Timeline: Visualization of Inter-Operator Parallelism

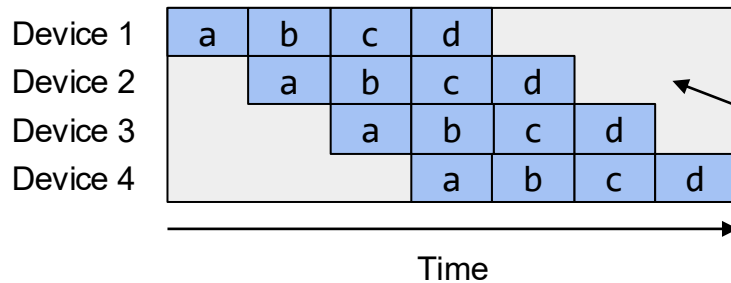


- Gray area ( indicates devices being idle (a.k.a. Pipeline bubbles).
- Only 1 device activated at a time.
- **Pipeline bubble percentage** = $\text{bubble_area} / \text{total_area}$
= $(D - 1) / D$, assuming D devices.

Reduce Pipeline Bubbles via Pipelining Inputs

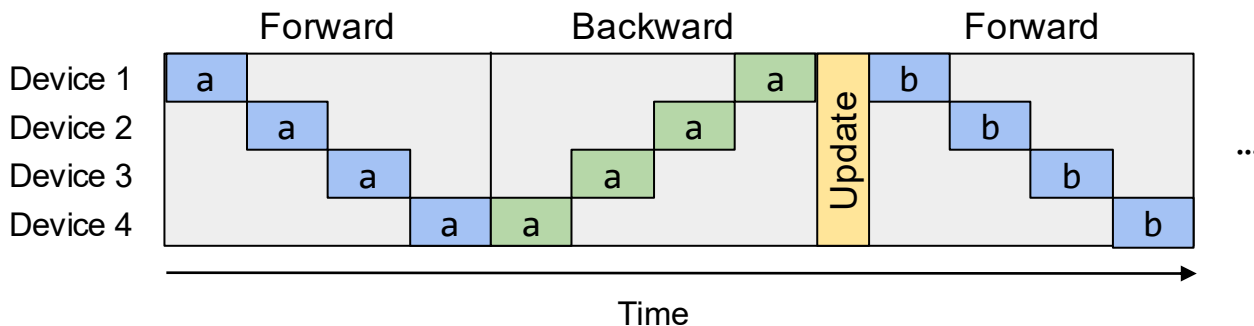
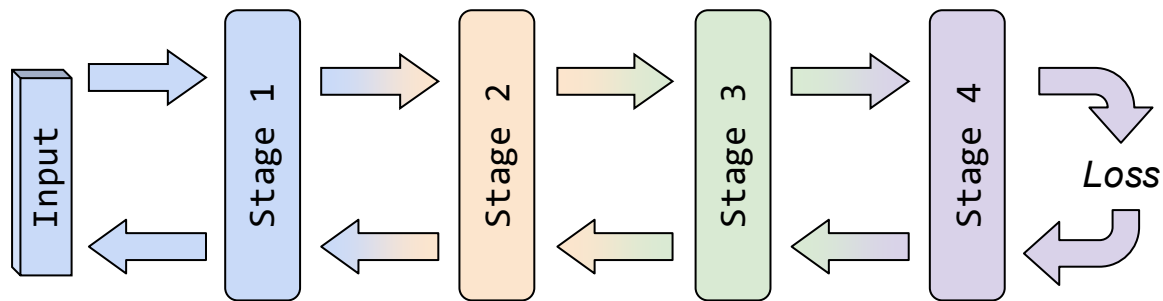


Used in inference.



Pipeline bubbles percentage
 $= (D - 1) / (D - 1 + N)$
with D devices and N inputs.

Training: Forward & Backward Dependency

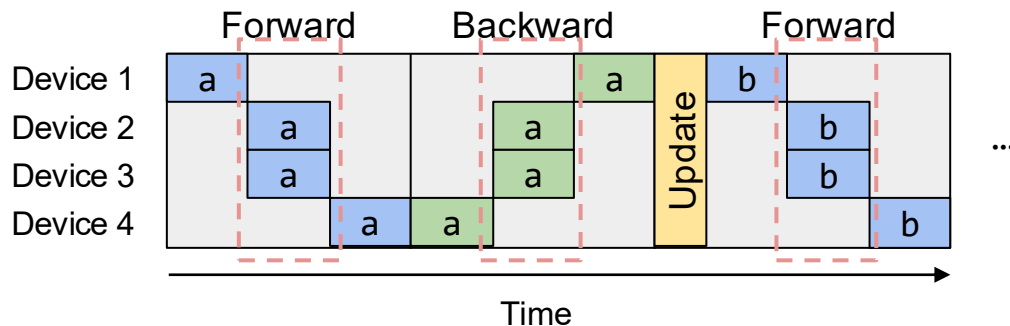
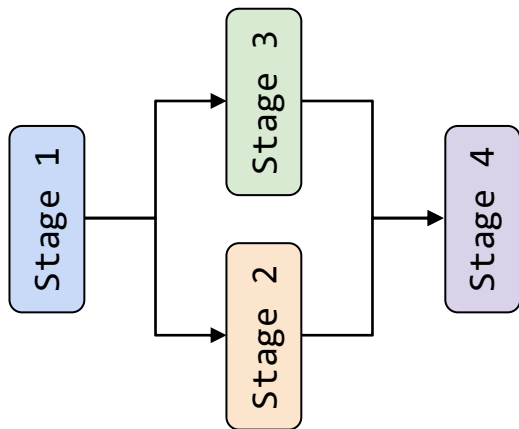


How to Reduce Pipeline Bubbles for Training?

- Device Placement
- Synchronous Pipeline Parallel Algorithms
 - GPipe
 - 1F1B
 - Interleaved 1F1B
 - TeraPipe
 - Chimera
- Asynchronous Pipeline Parallel Algorithms
 - AMPNet
 - Pipedream/Pipedream-2BW

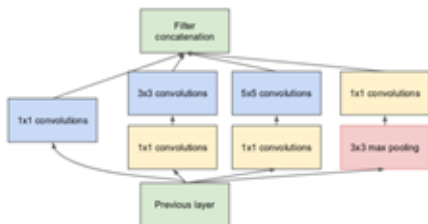
Device Placement

Idea: Slice the branches of a neural network into multiple stages so they can be calculated concurrently.

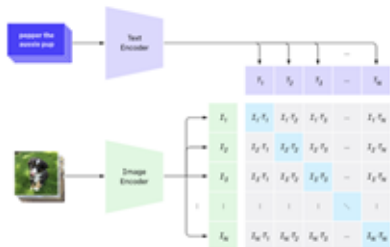


Device Placement: Limitations

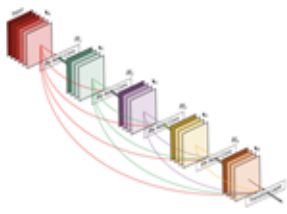
Only works for specific NNs with branches:



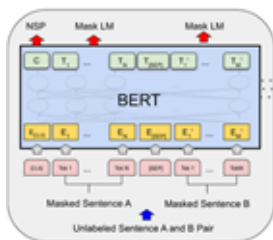
✓ Inception Module



✓ Contrastive Model

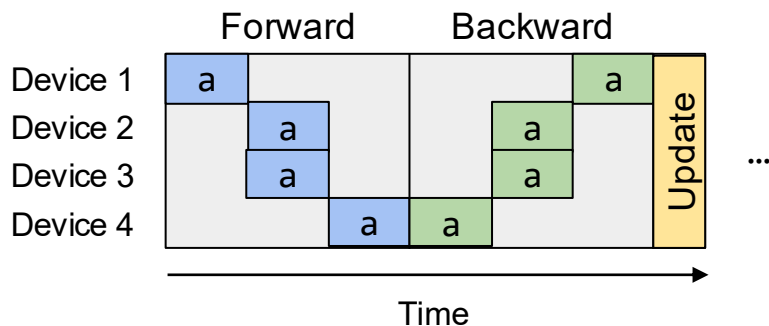


✗ Other ConvNets



✗ Transformers

Device Utilization is still low:



Note: device placement needs to be combined with the other pipeline schedules discussed later to further improve device utilization.

Synchronous Pipeline Parallel Schedule

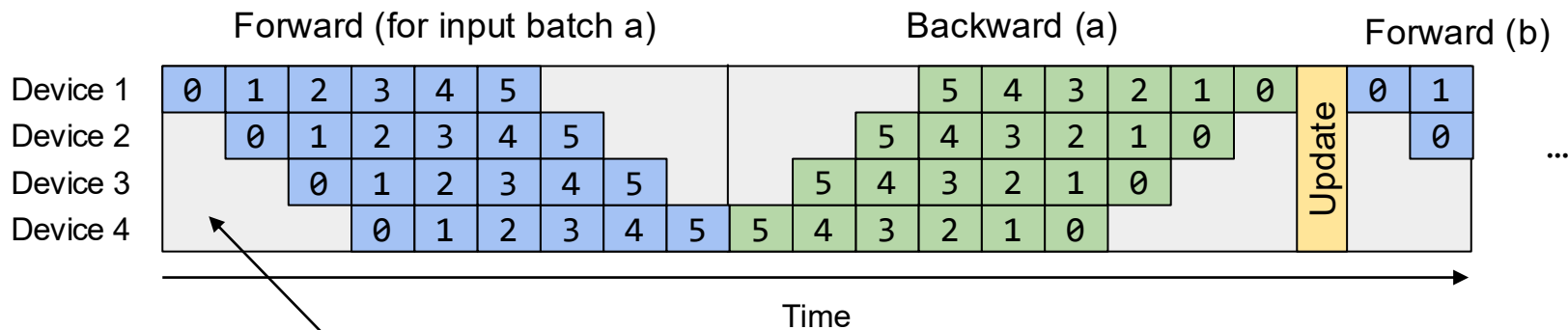
Idea: Modify pipeline schedule to improve efficiency, but keep the computation and convergence semantics exactly the same as if training with a single device.

GPipe

Idea: Partition the input batch into multiple “*micro-batches*”. Pipeline the micro-batches. Accumulate the gradients of the micro-batches:

$$\nabla L_{\theta}(x) = \frac{1}{N} \sum_{i=1}^N \nabla L_{\theta}(x_i)$$

Example: Slice each input batch into 6 micro-batches:



Pipeline bubbles percentage = $(D - 1) / (D - 1 + N)$
with D devices and N micro-batches.

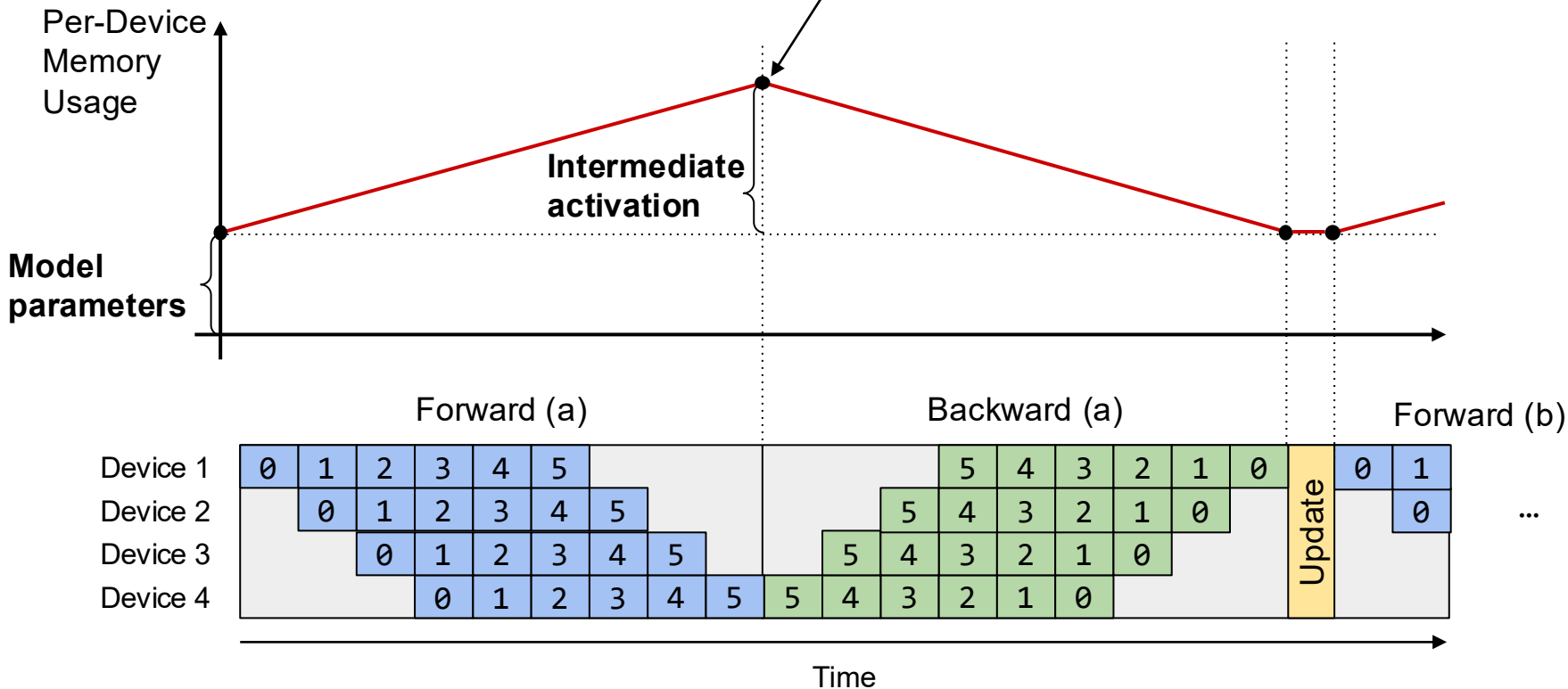
GPipe: Experimental Results

Table: Normalized training throughput using GPipe with different number of devices (stages) and different number of micro-batches M on TPUs.

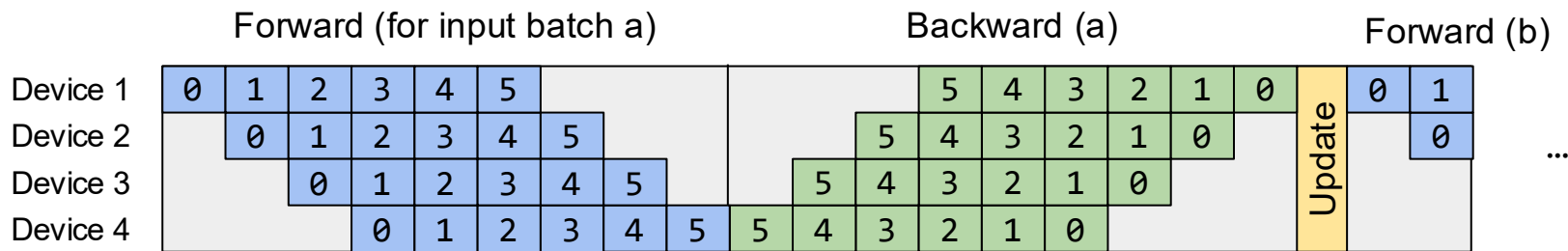
	#TPUs = 2	#TPUs = 4	#TPUs = 8
#Micro-batches = 1	1	1.07	1.3
#Micro-batches = 4	1.7	3.2	4.8
#Micro-batches = 32	1.8	3.4	6.3

GPipe: Memory Usage

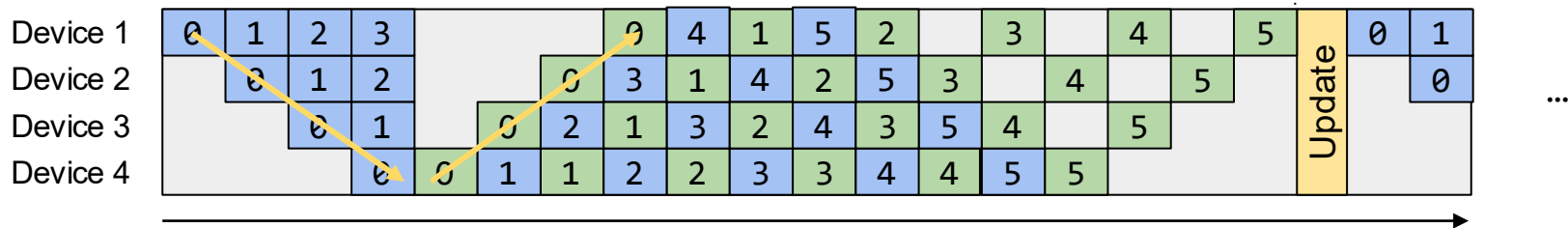
$$= \text{Parameters} + \text{Activation} \times \text{\#Micro-Batches}$$



GPipe Schedule:



1F1B (1 Forward 1 Backward) Schedule:



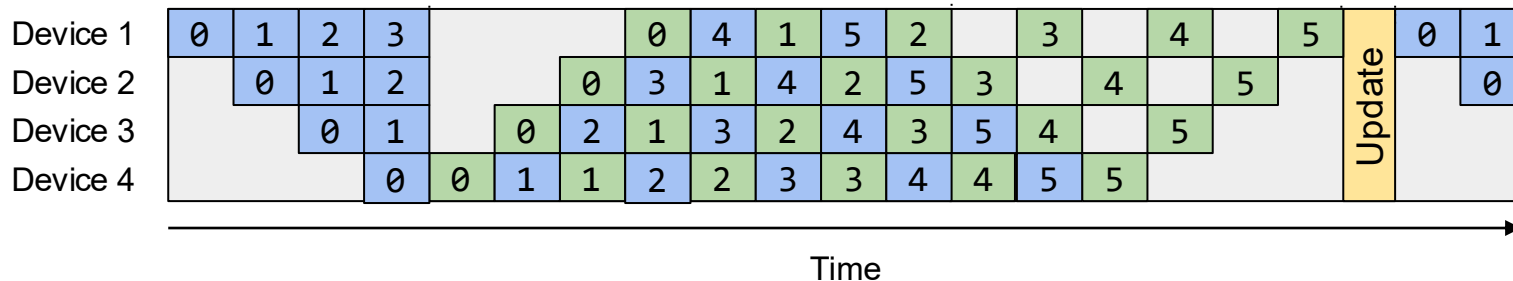
Same Latency

Perform backward as early as possible

1F1B Memory Usage

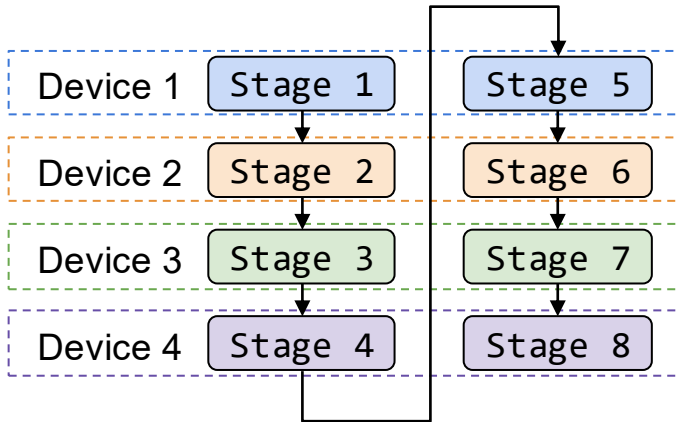
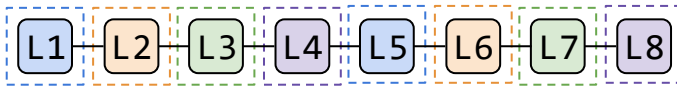
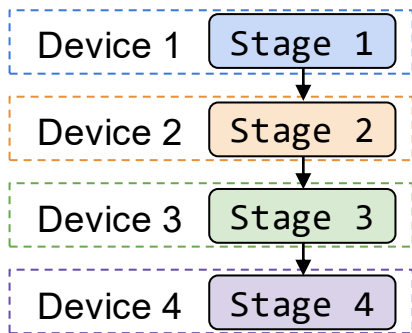
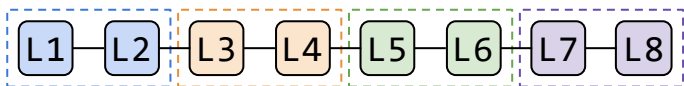
Maximum
per-device
memory
usage

$$= \text{Parameters} + \text{Activation} \times \text{\textcolor{red}{\#Micro-Batches}} \times \text{\textcolor{green}{\#Devices}}$$



Interleaved 1F1B

Idea: Slice the neural network into more fine-grained stages and assign multiple stages to reduce pipeline bubble.



Interleaved 1F1B

Pro:

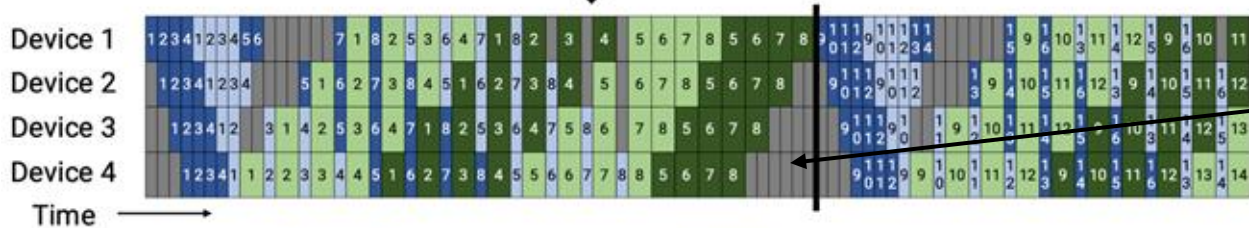
Higher pipeline efficiency with fewer pipeline bubbles.

Con:

More communication overhead between stages.



Assign multiple stages to each device



Forward Pass



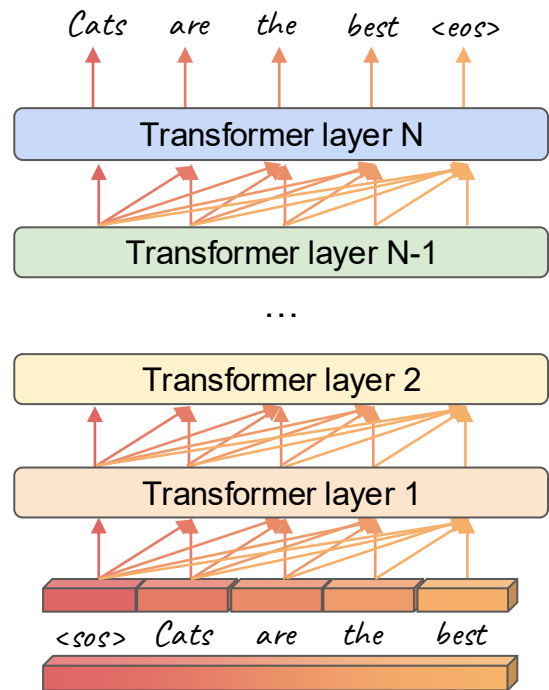
Backward Pass

Pipeline bubbles percentage
= $(D - 1) / (D - 1 + KN)$
with D devices, K stages on each
device, and N micro-batches.

TeraPipe

Idea: The computation of an input token only depends on previous tokens but not future tokens for autoregressive models.

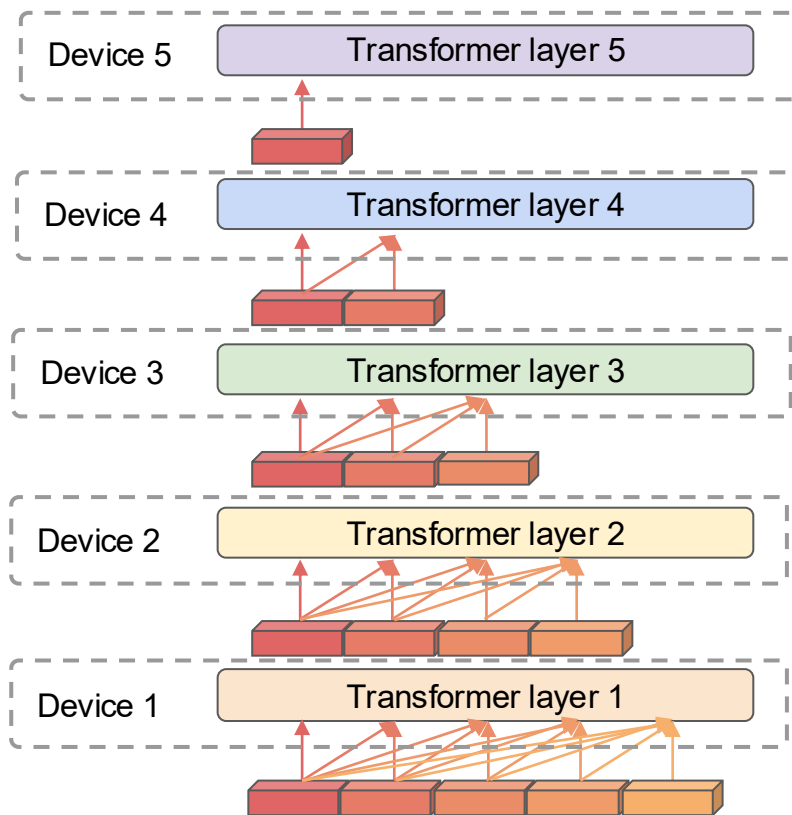
Further reduce the bubble size by pipelining within a sequence.



TeraPipe

Idea: The computation of an input token only depends on previous tokens but not future tokens for autoregressive models.

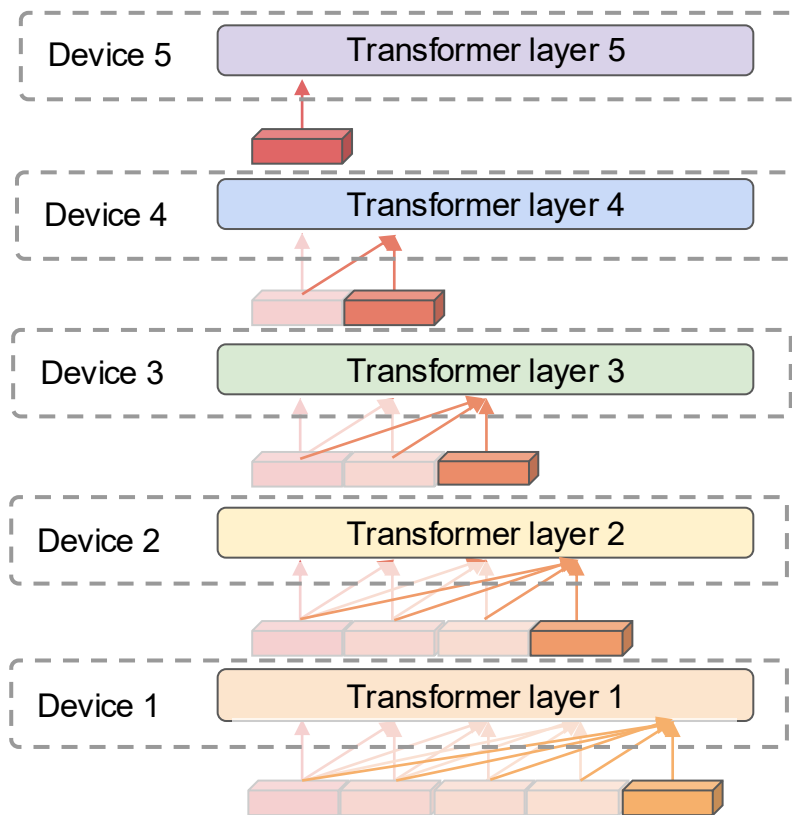
Further reduce the bubble size by pipelining within a sequence.



TeraPipe

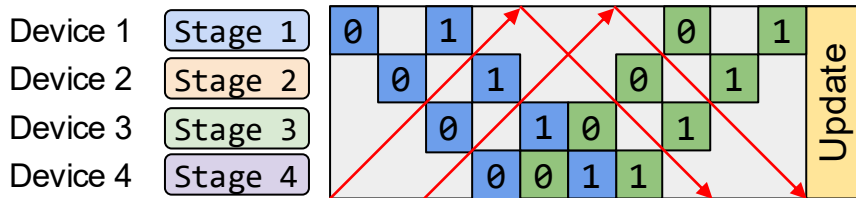
Idea: The computation of an input token only depends on previous tokens but not future tokens for autoregressive models.

Further reduce the bubble size by pipelining within a sequence.

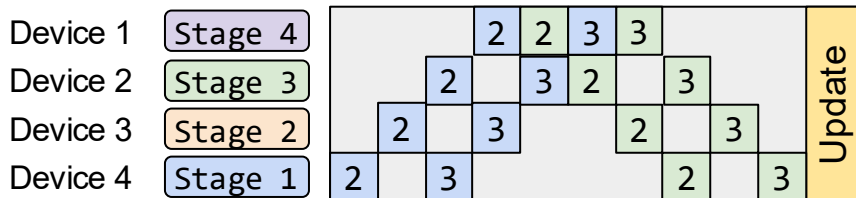


Chimera

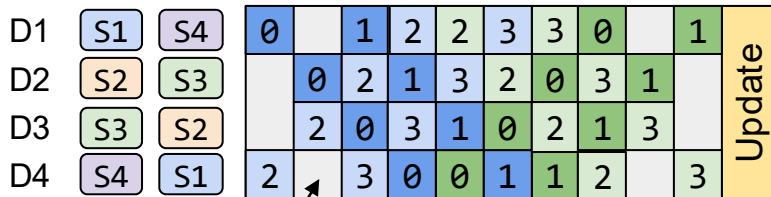
Idea: Store bi-directional stages and combine bidirectional pipeline to further reduce pipeline bubbles.



+



Extra copy of parameters & extra synchronization.



Pipeline bubbles percentage

$$= (D - 2) / (D - 2 + 2N)$$

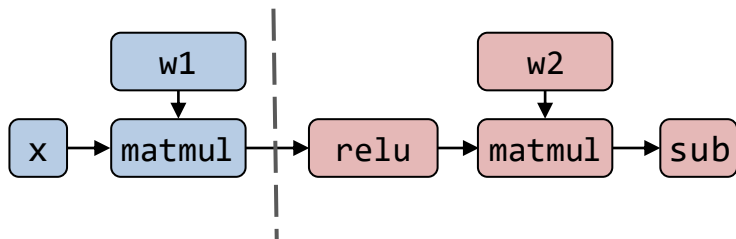
 with D devices and N micro-batches.

Where We Are

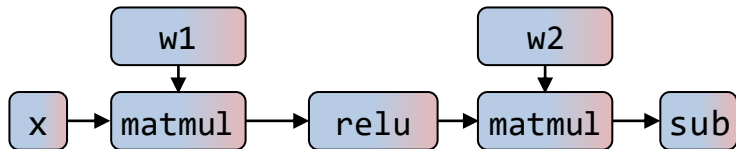
- Why Parallelization: Technology Trend
- ML Parallelism Overview
- Collective Communication Overview
- Inter and intra-op parallelism
 - Inter-op parallelism
 - **Intra-op parallelism**

Recap: Intra-op and Inter-op

Strategy 1: Inter-operator Parallelism



Strategy 2: Intra-operator Parallelism



This section:

1. How to parallelize an **operator** ?
2. How to parallelize a **graph** ?

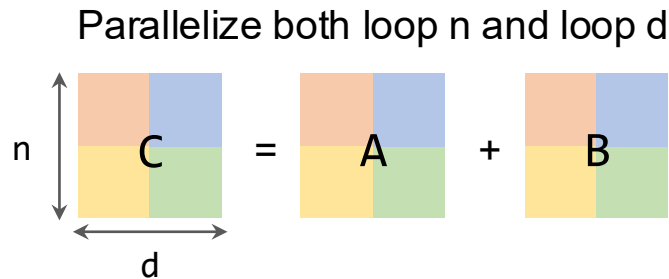
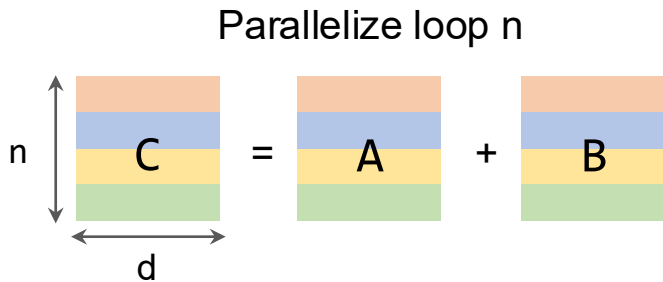
Parallelize One Operator

Element-wise operators

```
for n in range(0, N):  
    for d in range(0, D):  
        C[n,d] = A[n,d] + B[n,d]
```

No dependency on the two for-loops.
Can arbitrarily split the for-loops on different devices.

device 1 device 2 device 3 device 4



a lot of
other variants
...

Parallelize One Operator

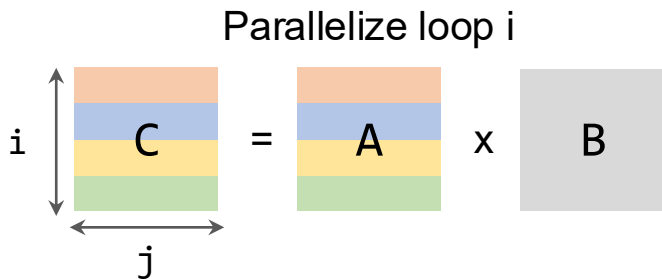
Matrix multiplication

```
for i in range(0, N):  
    for j in range(0, M):  
        for k in range(0, K):  
            C[i,j] = C[i,j] + A[i,k] x B[k,j]
```

No dependency on the two spatial for-loops.
Can arbitrarily split the for-loops on different devices.

Accumulation on this reduction loop.
Have to accumulate partial results if we split
this for-loop

device 1 device 2 device 3 device 4 replicated



$$\begin{bmatrix} C_1 \\ C_2 \\ C_3 \\ C_4 \end{bmatrix} = \begin{bmatrix} A_1 \\ A_2 \\ A_3 \\ A_4 \end{bmatrix} \times B$$

Parallelize One Operator

Matrix multiplication

```
for i in range(0, N):  
    for j in range(0, M):  
        for k in range(0, K):  
            C[i,j] = C[i,j] + A[i,k] x B[k,j]
```

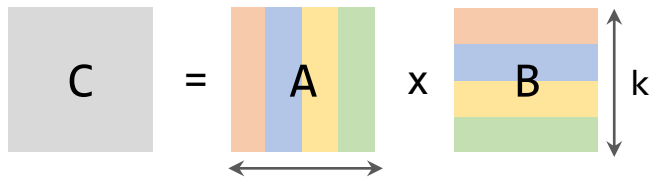
No dependency on the two spatial for-loops.
Can arbitrarily split the for-loops on different devices.

Accumulation on this reduction loop.

Have to accumulate partial results if we split
this for-loop

device 1 device 2 device 3 device 4 replicated

Parallelize loop k



(got by all-reduce) k

$$C = [A_1 \ A_2 \ A_3 \ A_4] \begin{bmatrix} B_1 \\ B_2 \\ B_3 \\ B_4 \end{bmatrix} = A_1 B_1 + A_2 B_2 + A_3 B_3 + A_4 B_4$$

Parallelize One Operator

Matrix multiplication

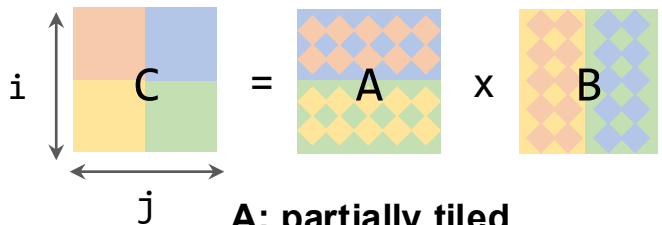
```
for i in range(0, N):  
    for j in range(0, M):  
        for k in range(0, K):  
            C[i,j] = C[i,j] + A[i,k] x B[k,j]
```

No dependency on the two spatial for-loops.
Can arbitrarily split the for-loops on different devices.

Accumulation on this reduction loop.
Have to accumulate partial results if we split
this for-loop

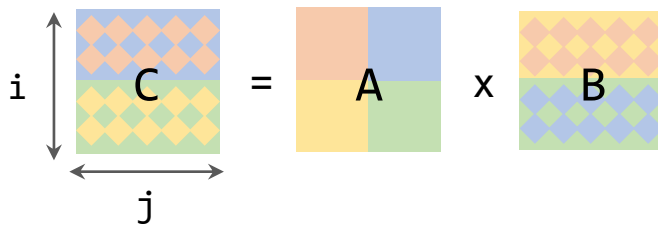
device 1 device 2 device 3 device 4

Parallelize loop i and j



Device 1 and 2 hold a replicated tile
Device 3 and 4 hold a replicated tile

Parallelize loop i and k



a lot of
other variants
...

Parallelize One Operator

2D Convolution

```
for n in range(0, N):
    for co in range(0, CO):
        for h in range(0, H):
            for w in range(0, W):
                for ci in range(0, CI):
                    for kh in range(0, KH):
                        for kw in range(0, KW):
                            C[n,co,h,w] += A[n,co,h+kh,w+kw] x B[kh,kw,co,ci]
```

Simple spatial loops. Can be arbitrarily split.

Stencil computation loops. Splitting these requires careful boundary handling.

Reduction loop. Need to accumulate partial results.

Reduction loops. But usually too small (≤ 5) for parallelization.

Simple case: Parallelize loop n , co , ci , then the parallelization strategies are almost the same as matmul's.

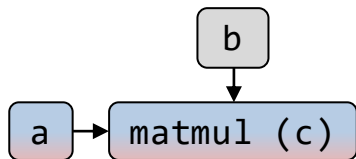
Complicated case: Parallelize loop h and w

Data Parallelism as A Case of Intra-op Parallelism

 Replicated  Row-partitioned  Column-partitioned

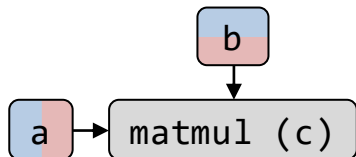
Matmul Parallelization Type 1

communication cost = 0



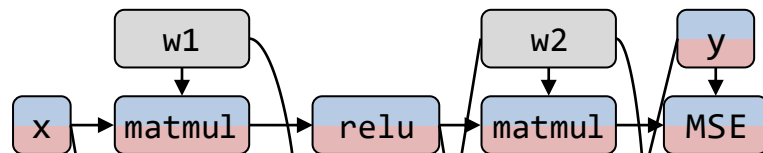
Matmul Parallelization Type 2

communication cost = all-reduce(c)



Forward Pass

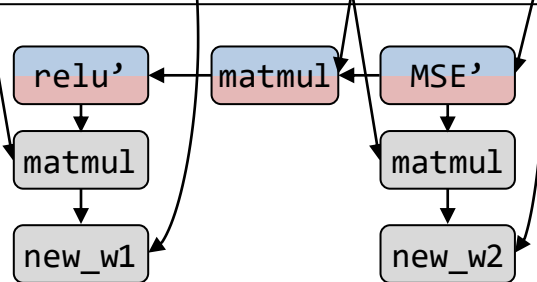
Two “Type 1” matmuls: no communication



Backward Pass

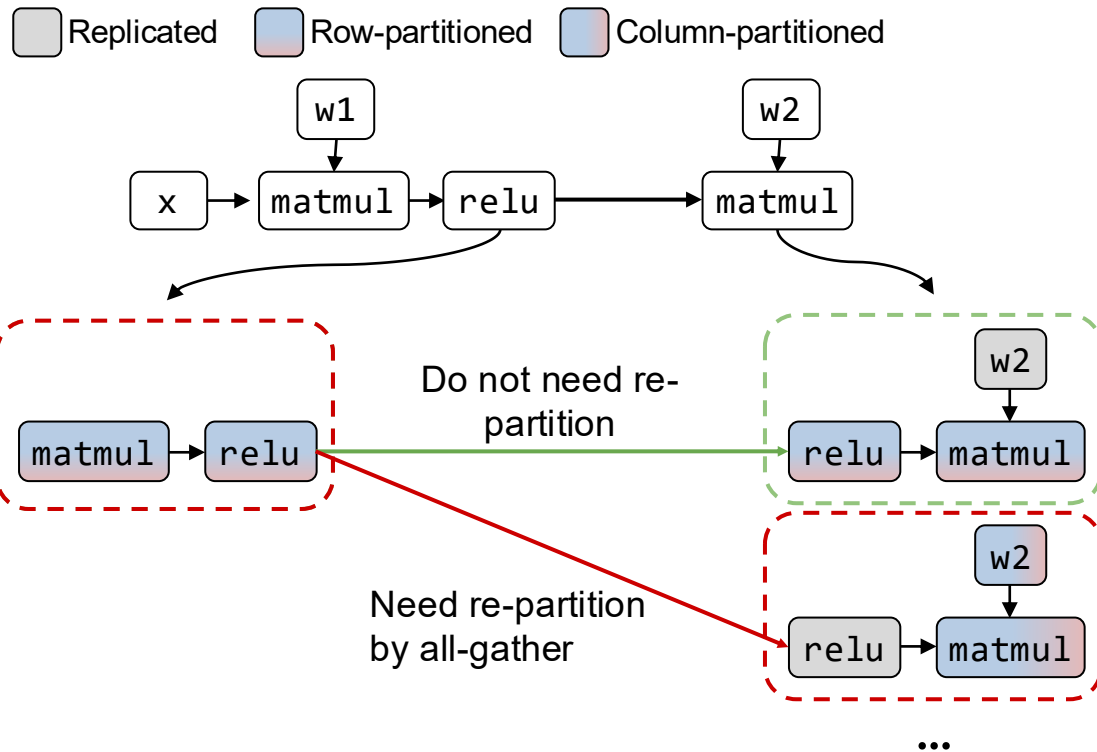
One “Type 1” matmul: no communication

Two “Type 2” matmuls: require all-reduce



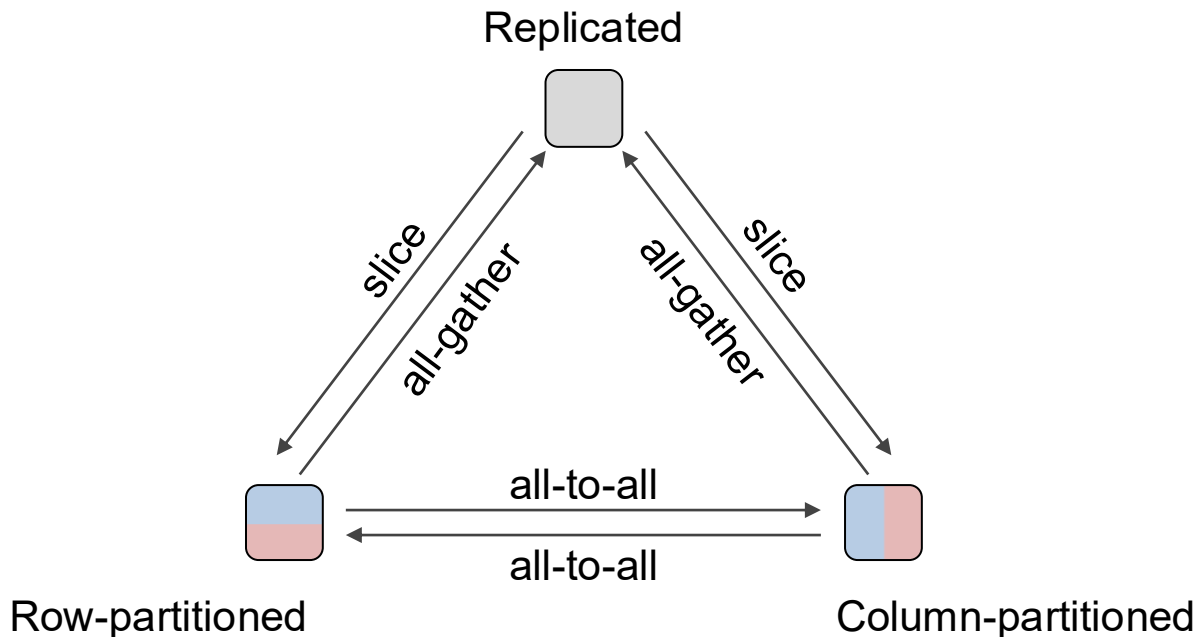
Re-partition Communication Cost

Different operators' parallelization strategies require different partition format of the same tensor



Re-partition Communication Cost

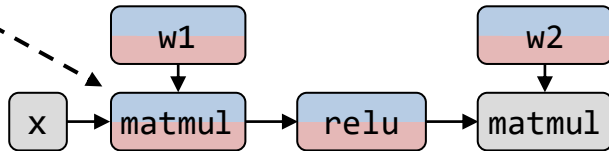
Different operators' parallelization strategies require different partition format of the same tensor



Parallelize All Operators in a Graph

Problem

Pick a parallel strategy
of each operator



Minimize **Node costs** (computation + communication) + **Edge costs** (re-partition communication)

Solution

- Manual design
- Randomized search
- Dynamic programming
- Integer linear programming

Important Projects

Model-specific Intra-op Parallel Strategies

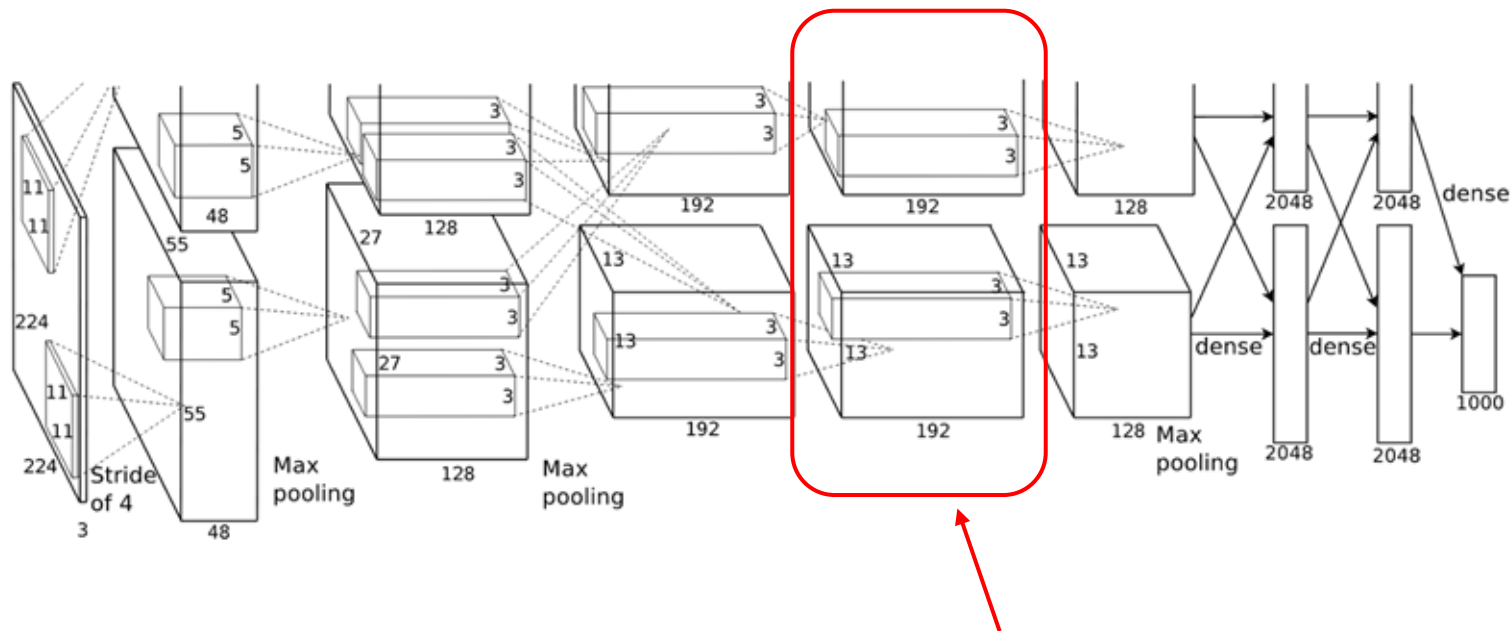
- AlexNet
- Megatron-LM
- GShard MoE

Systems for Intra-op Parallelism

- ZeRO
- Mesh-Tensorflow
- GSPMD
- Tofu
- FlexFlow

AlexNet

Result: increase top-1 accuracy by 1.7%

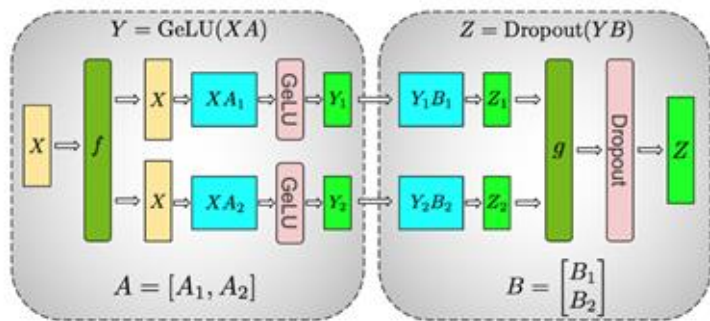


Assign a group convolution layer to 2 GPUs

Megatron-LM

Result: a large language model with 8.3B parameters that outperforms SOTA results

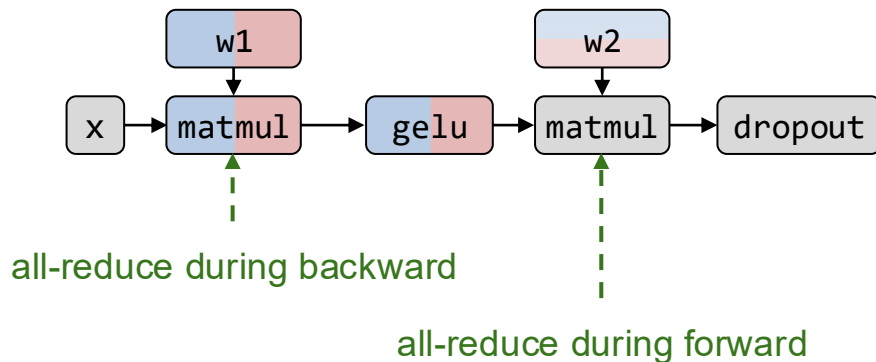
Figure 3 from the paper :
How to partition the MLP in the transformer.



(a) MLP

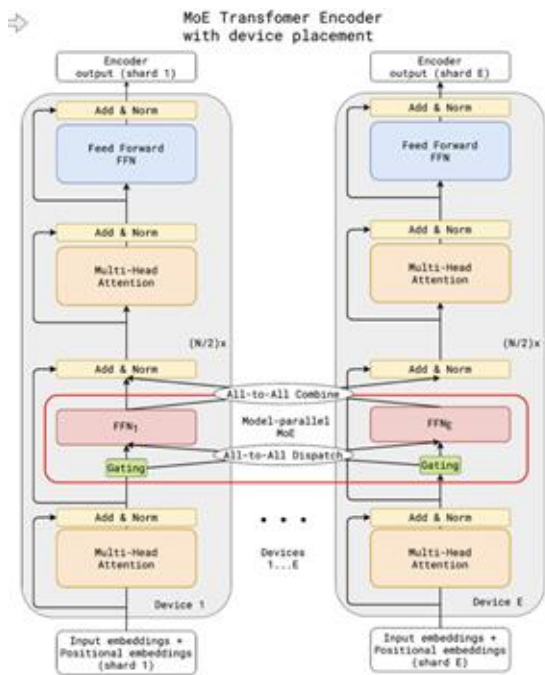
Illustrated with the notations in this tutorial

Replicated Row-partitioned Column-partitioned



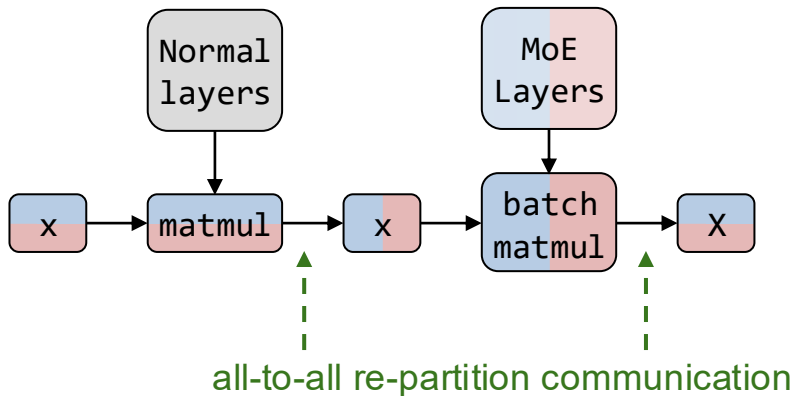
GShard MoE

Result: a multi-language translation model with 600B parameters that outperforms SOTA



Illustrated with the notations in this tutorial

Replicated Row-partitioned Expert-partitioned



ZeRO Optimizer

Problem

Data parallelism replicates gradients, optimizer states and model weights on all devices.

Idea

Partition gradients, optimizer states and model weights.

M is the number of parameters, N is the number of devices.

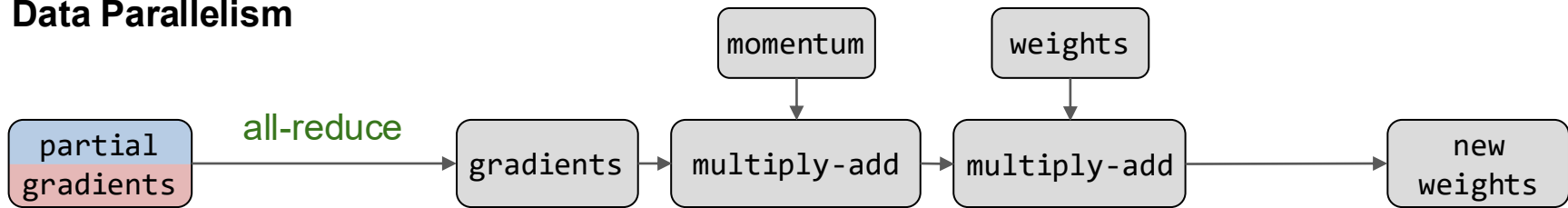
	Optimizer States (12M)	Gradients (2M)	Model Weights (2M)	Memory Cost	Communication Cost
Data Parallelism	Replicated	Replicated	Replicated	$16M$	all-reduce(2M)
ZeRO Stage 1	Partitioned	Replicated	Replicated	$4M + \frac{12M}{N}$	all-reduce(2M)
ZeRO Stage 2	Partitioned	Partitioned	Replicated	$2M + \frac{14M}{N}$	all-reduce(2M)
ZeRO Stage 3	Partitioned	Partitioned	Partitioned	$\frac{16M}{N}$	1.5 all-reduce(2M)

ZeRO Stage 2

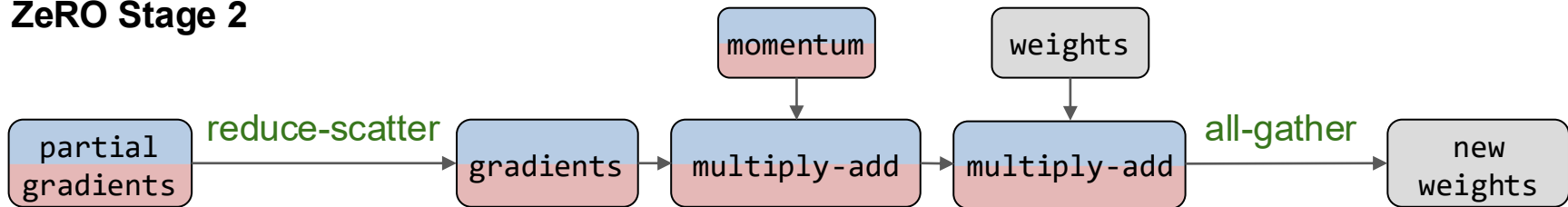
Key Idea: all-reduce = reduce-scatter + all-gather

 Replicated  Partitioned

Data Parallelism



ZeRO Stage 2



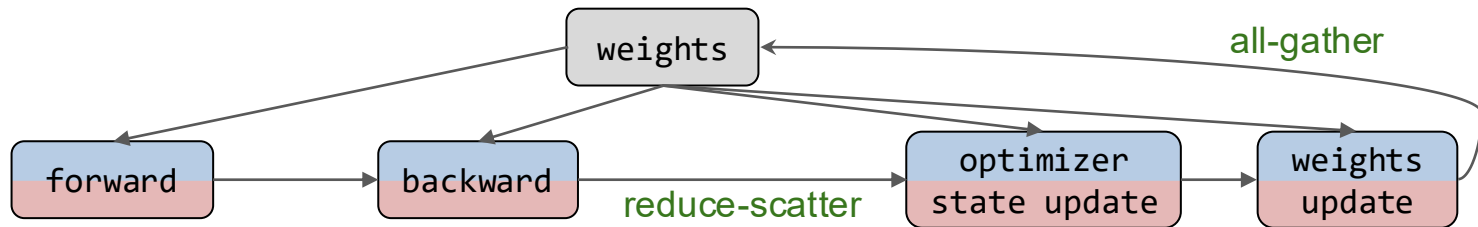
Same communication cost but save memory by partitioning more tensors

ZeRO Stage 3

 Replicated  Partitioned

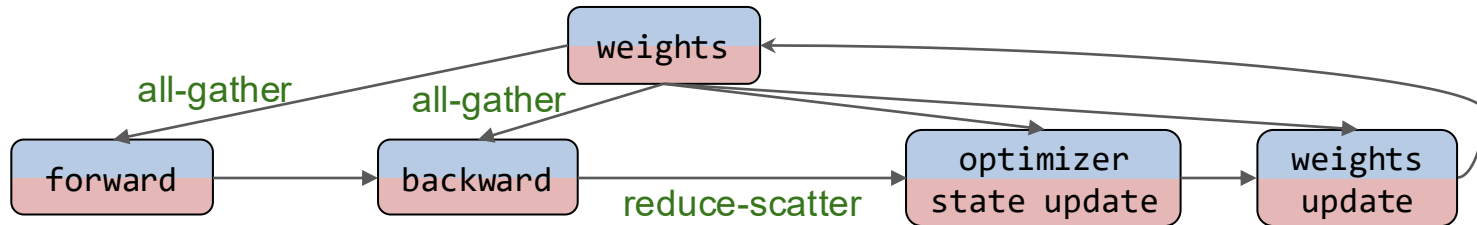
ZeRO Stage 2

communication cost
= all-reduce

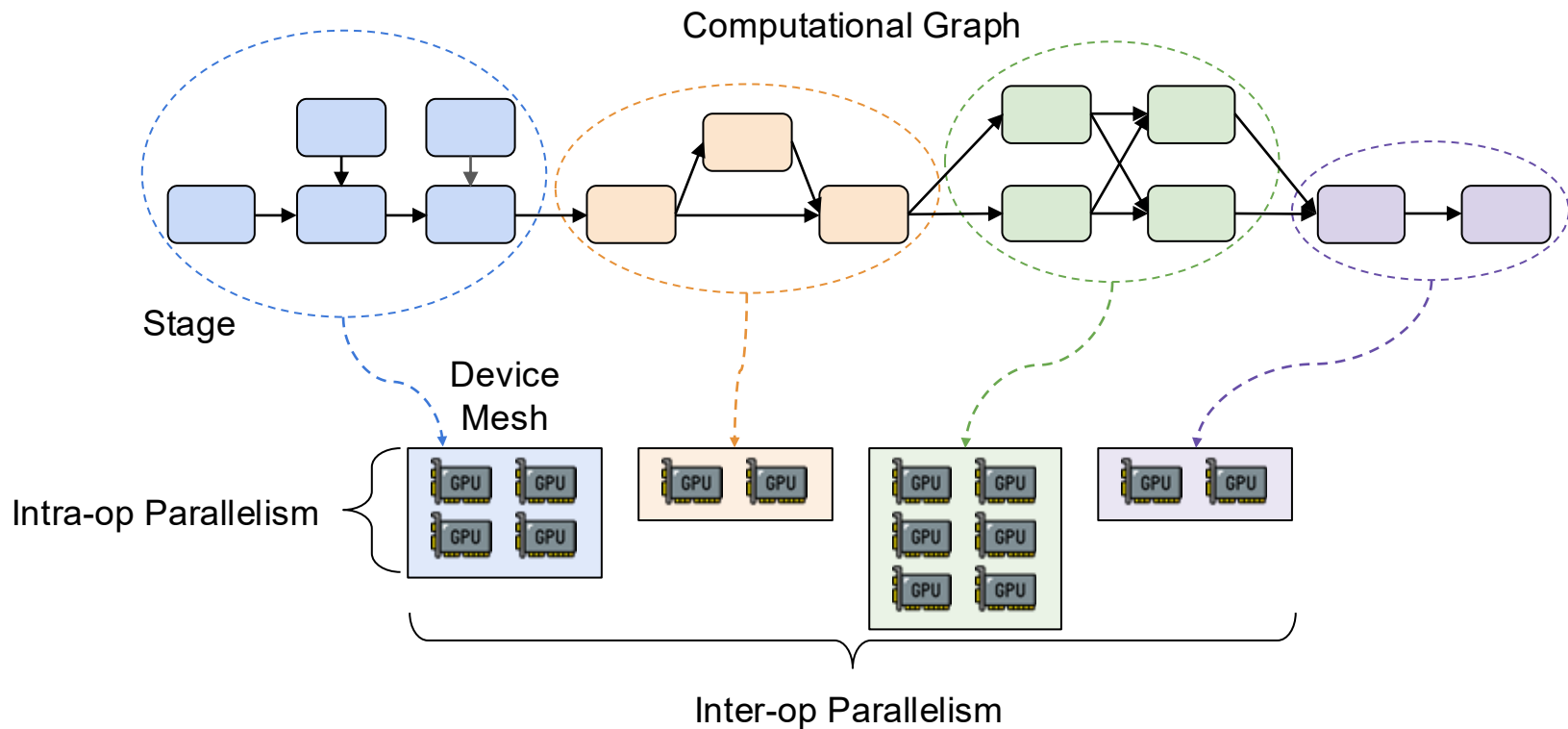


ZeRO Stage 3

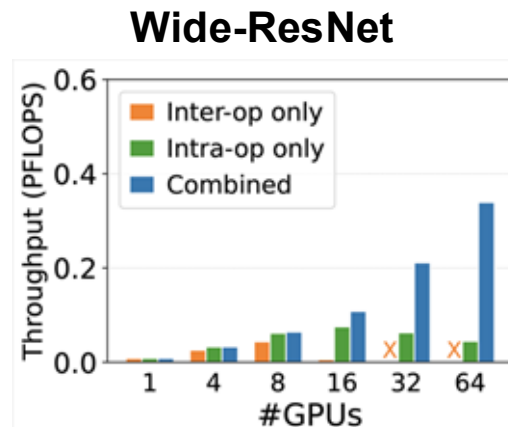
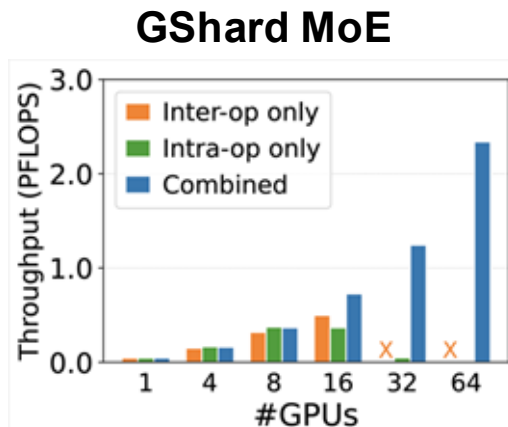
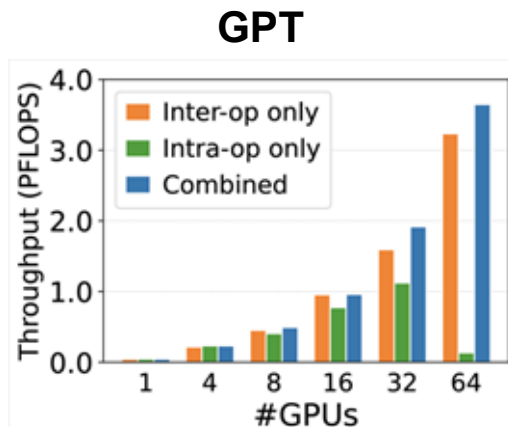
communication cost
= 1.5 all-reduce



Combine Intra-op Parallelism and Inter-op Parallelism



Combine Intra-op Parallelism and Inter-op Parallelism



Combining inter- and intra-operator parallelism scales to more devices.

Intra-operator Parallelism Summary

- We can parallelize a single operator by exploiting its internal parallelism
- To do this for a whole computational graph, we need to choose strategies for all nodes in the graph to minimize the communication cost
- Intra-op and inter-op can be combined

Other Techniques for Training Large Models

System-level Memory Optimizations

- Rematerialization/Gradient Checkpointing
- Swapping

ML-level Optimizations

- Quantization
- Sparsification
- Low-rank approximation

Chen, Tianqi, et al. "Training deep nets with sublinear memory cost." *arXiv 2016*

Rajbhandari, Samyam, et al. "Zero-infinity: Breaking the gpu memory wall for extreme scale deep learning." *SC 2021*.

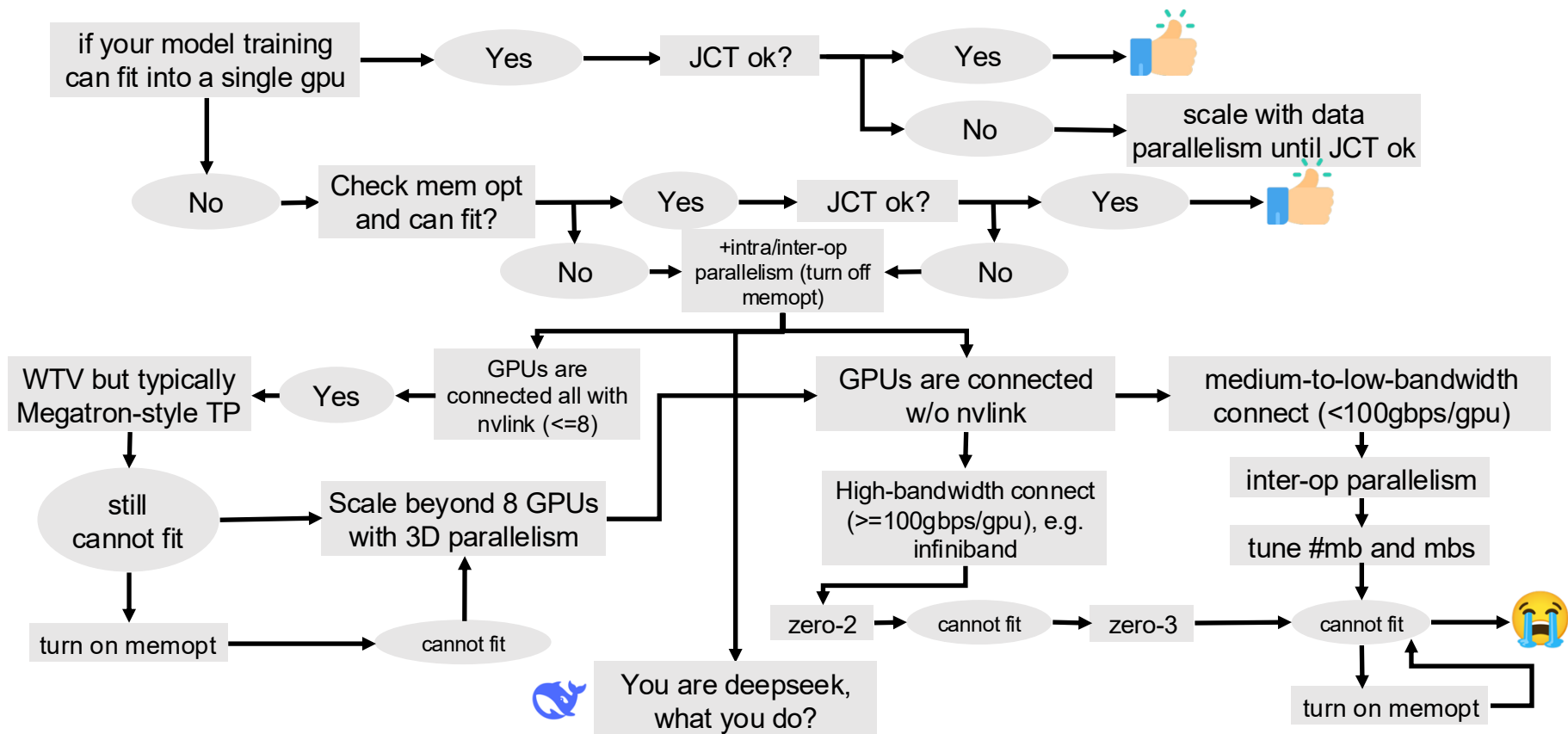
Tang, Hanlin, et al. "1-bit adam: Communication efficient large-scale training with adam's convergence speed." *ICML 2021*.

Shazeer, Noam, and Mitchell Stern. "Adafactor: Adaptive learning rates with sublinear memory cost." *ICML 2018*.

Summary: How to Choose Parallelism

1. Use automatic compiler if not transformer
2. Manual parallelism search for transformers:
 - Factors to consider
 - #GPUs you have
 - Model size
 - JCT (Job completion time)
 - Communication bandwidth
 - etc.

Hao's Ultimate Guide



The End

- The world now is changing 10x faster than before
- Innovations happen 10x faster
 - What you have learned can be replaced in 1 – 2 years
 - This will become a **norm**
- What we really hope you learn from this course:
 - Ability to identify the right problems
 - Ability to understand “trends”
 - Ability to “predict the future” (I hope so)